

Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #11

**Behavioral Control with Finite State Machines. Part 1:
Button-LED FSM**

Author:

Alexandru Cebotari

Group FAF-233

Verified by:

Alexei Martiniuc

Technical task

Purpose

This report documents the implementation of Lab 11 according to the provided condition. The requested scope is Lab 7, Part 1 only: a Button-LED application controlled by a finite state machine. The system has two states, `LED_OFF` and `LED_ON`; each valid button press toggles the current state. A software debounce filter rejects contact bounce, and the current FSM state is reported through serial `STDIO` and `LCD1602`.

Objectives

- implement a modular Moore finite state machine for Button-LED control;
- detect only valid button press events and ignore button release events;
- implement software debounce to prevent false state transitions;
- separate the FSM logic from button, LED, serial, and LCD driver code;
- report state changes and periodic status through `printf` and `LCD1602`;
- provide Wokwi simulation files and PlatformIO build support for Arduino Uno;
- keep the solution limited to Part 1 and exclude the intelligent traffic light from Part 2.

Individual task and interpretation

The final project uses Arduino Uno, one push button connected to D12 with the internal pull-up resistor enabled, one LED connected to D9 through a resistor, and `LCD1602` using the same 4-bit connections as Lab 10. The implementation intentionally does not use FreeRTOS because Part 1 can be satisfied with a non-blocking sequential loop. The selected debounce configuration samples the button every 5 ms and requires 6 consecutive equal samples, producing a stable transition after approximately 30 ms, which remains below the 100 ms latency requirement.

1 1. Domain analysis

1.1 1.1 Motivation

Finite state machines are useful in embedded systems because they describe behavior with a limited number of states, inputs, outputs, and transitions. This approach is more explicit than ad-hoc conditional logic: every possible input in every state has a defined result. For a Button-LED application, the FSM model is appropriate because the LED can be either off or on, and the only meaningful external event is a valid button press.

1.2 1.2 Moore FSM context

The implemented solution uses a Moore FSM. In a Moore machine, the output depends only on the current state. This is suitable for LED control because the LED state should remain stable while the button is held or released. The button press is used only as a transition event, not as a direct LED output.

1.3 1.3 Hardware and software context

- Hardware platform: Arduino Uno.
- Input subsystem: push button on D12, configured as `INPUT_PULLUP`, active LOW.
- Output subsystem: LED on D9, active HIGH through a current-limiting resistor.
- Reporting subsystem: USB serial at 115200 baud and LCD1602 in 4-bit mode.
- LCD pins: RS=D7, E=D6, D4=D5, D5=D4, D6=D3, D7=D13.
- Development resources: PlatformIO, Arduino framework, Wokwi, LaTeX, PlantUML.

1.4 1.4 Problem definition

The problem is to design and implement a modular embedded application where a debounced button press toggles a LED through a finite state machine. The system must avoid repeated toggles during one physical press, ignore release transitions, respond in less than 100 ms, and expose the current state through serial and LCD interfaces.

Section conclusion

The domain analysis justifies the use of a Moore FSM and a debounce filter for a deterministic Button-LED application with clear input, output, and state-transition behavior.

2 2. Conceptualization and design

2.1 2.1 Design motivation

The design stage converts the laboratory condition into technical requirements, architecture, behavioral diagrams, and validation criteria. This keeps traceability between the condition, implementation, and report as required by the project documentation guideline.

2.2 2.2 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall read the button state from a digital input pin.	F	raw button state is sampled periodically
R2	The application shall debounce the button signal before creating events.	F	bounce does not create extra press events
R3	The application shall implement a two-state FSM with <code>LED_OFF</code> and <code>LED_ON</code> .	F	FSM table contains both states
R4	The application shall toggle the LED state once for each valid button press.	F	one press produces one state transition
R5	The application shall ignore button release events for LED toggling.	F	release is reported but does not change state
R6	The application shall report current state and relevant events through serial <code>STDIO</code> and <code>LCD1602</code> .	F	status/event lines and LCD state are updated
R7	The application shall respond to a valid press in less than 100 ms.	NF	debounce latency is about 30 ms
R8	The code shall be modular, separating drivers, services, and application control.	NF	source tree follows layered decomposition
R9	The project shall build for Arduino Uno in <code>PlatformIO</code> .	NF	<code>pio run</code> succeeds
R10	The implementation shall remain limited to Lab 7 Part 1 only.	NF	no traffic-light Part 2 logic is included

Table 1: Technical requirements for Lab 11 Part 1

2.3 2.3 System components and roles

Component	Role in the application
Push button	Generates the user input. It is connected to D12 and read as active LOW.
Debounce filter	Converts the noisy raw button signal into stable press and release transitions.
Button-LED FSM	Defines the two states and the transition table used to toggle the LED.

LED driver	Applies the FSM output to the LED connected on D9.
Serial STDIO driver	Redirects <code>printf</code> output to the Arduino serial interface.
LCD1602 display	Shows the current FSM state, LED state, button state, and toggle counter.
Application controller	Initializes modules and schedules button sampling and reporting.

2.4 2.4 Structural architectural design

The software is organized into three practical layers: the application controller, service modules, and hardware-facing drivers. The service layer contains reusable logic that does not directly access microcontroller pins, while the driver layer encapsulates Arduino GPIO and serial operations.

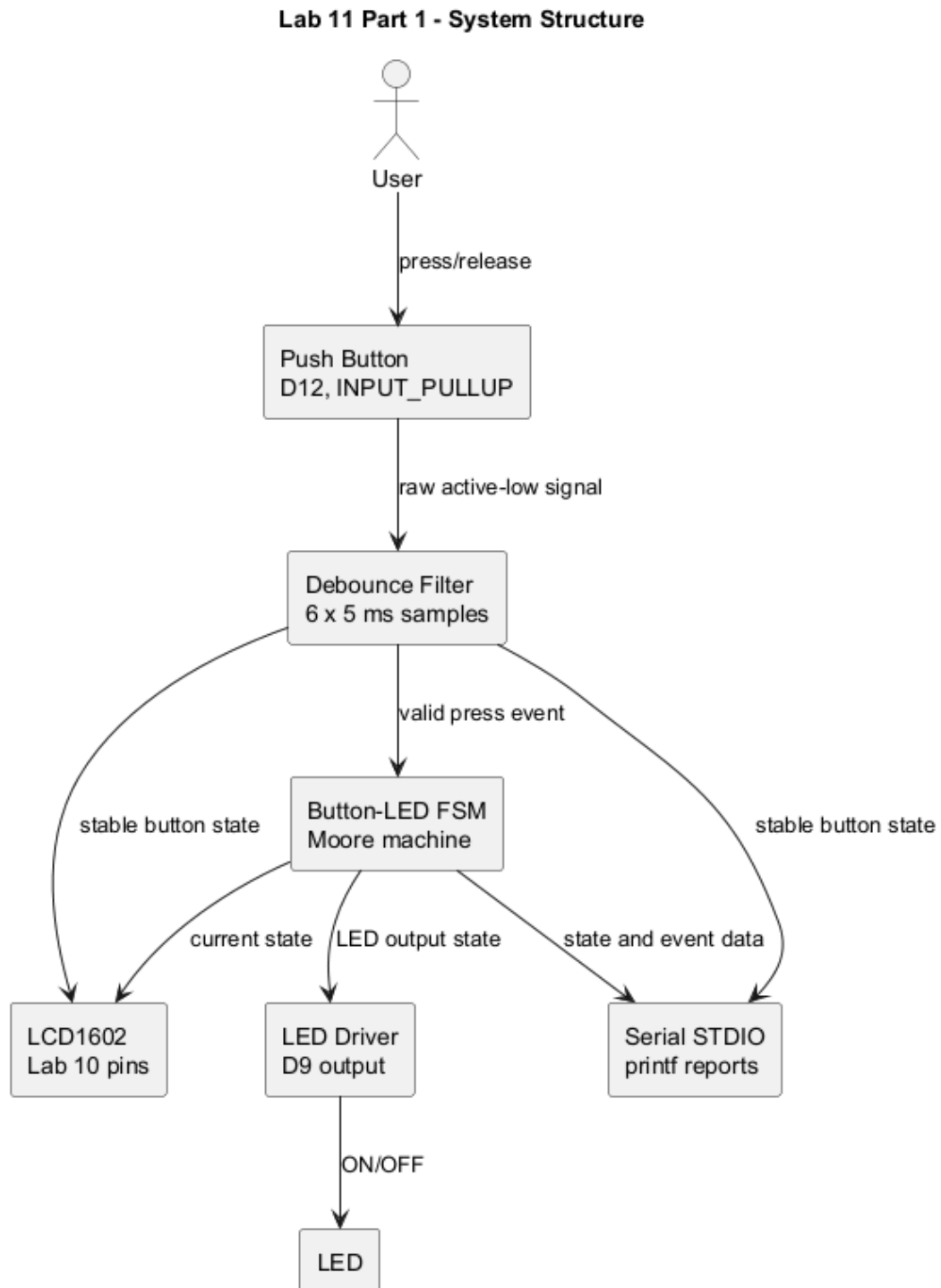


Figure 1: System structure for the Lab 11 Button-LED FSM application

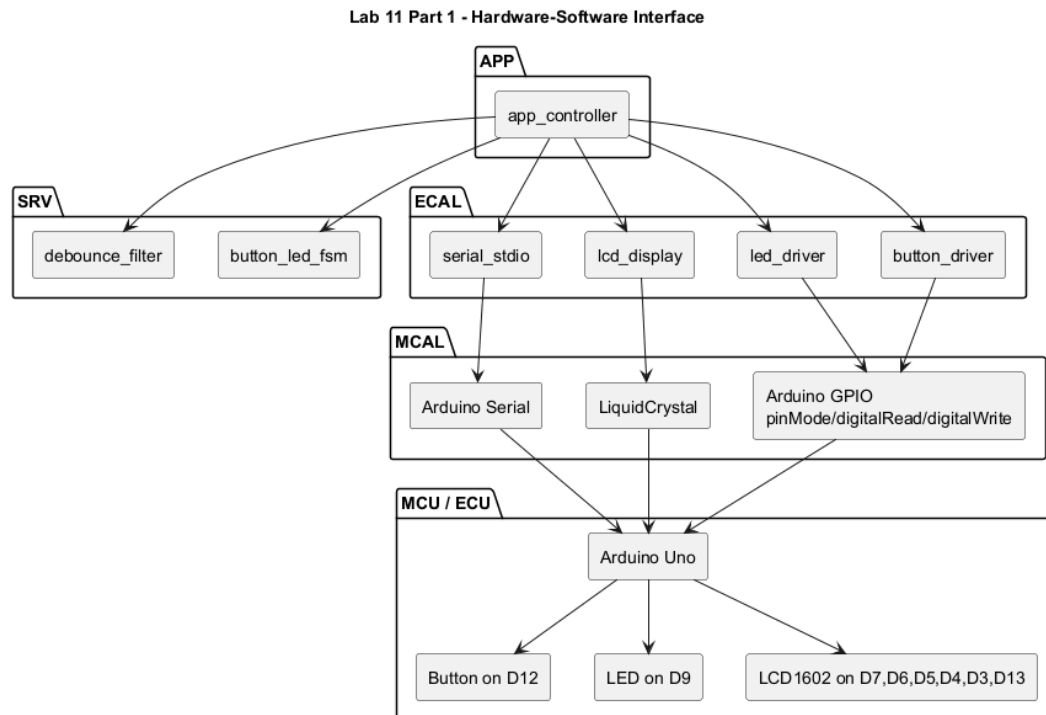


Figure 2: Hardware-software interface and layered decomposition

2.5 Behavioral modeling

The FSM is represented by a table where each row defines one state, the Moore output associated with the state, and the next state for each input event.

State ID	Name	Output	NoEvent	PressEvent
0	LED_OFF	LED LOW	LED_OFF	LED_ON
1	LED_ON	LED HIGH	LED_ON	LED_OFF

Table 3: FSM transition table for the Button-LED application

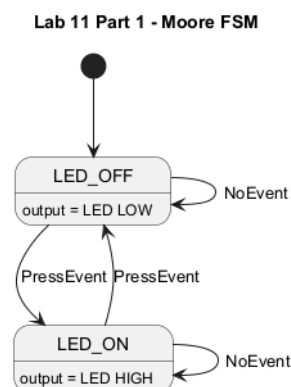


Figure 3: State diagram of the Button-LED Moore FSM

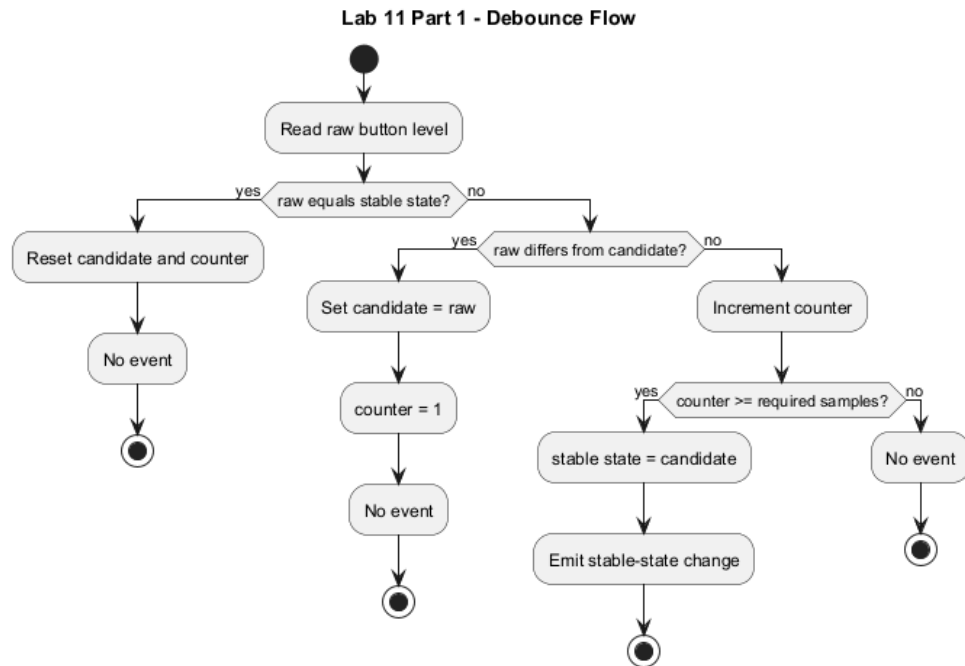


Figure 4: Debounce algorithm used to validate button transitions

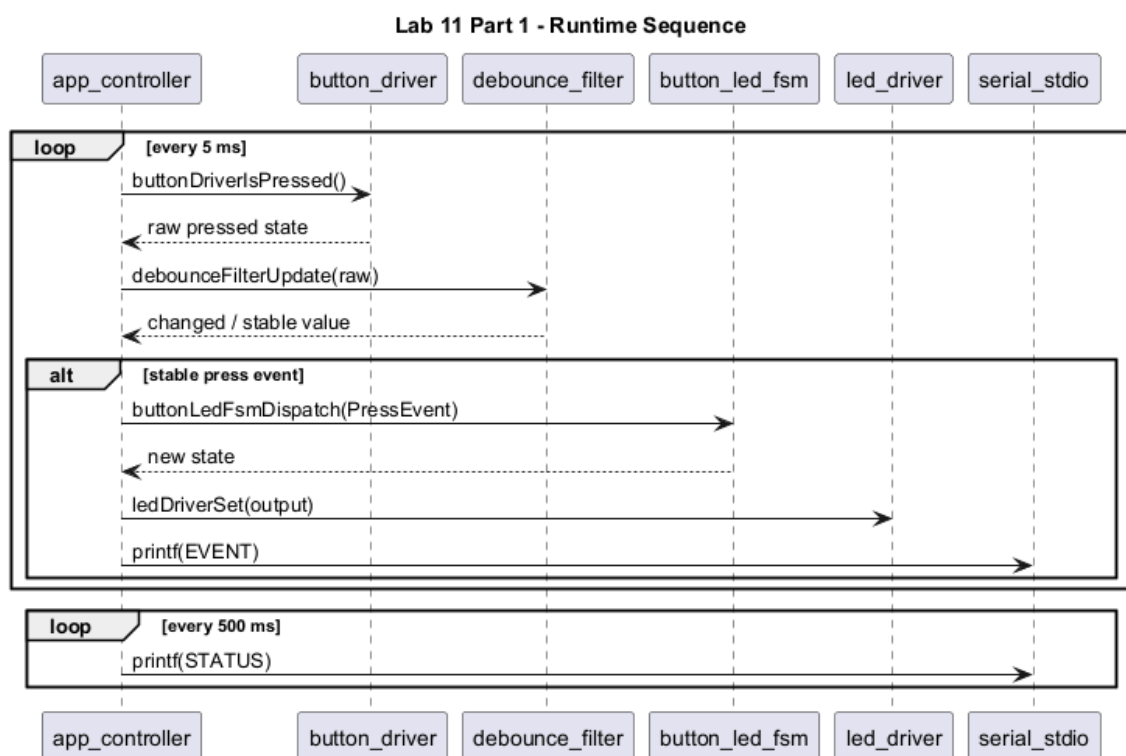


Figure 5: Runtime sequence for input sampling, FSM dispatch, and reporting

2.6 Test scenarios and validation criteria

ID	Scenario	Req.	Pass criterion
T1	Build the project with <code>pio run</code> .	R9	build succeeds for Arduino Uno
T2	Press the button once while LED is OFF.	R1-R4	LED changes to ON once
T3	Press the button once while LED is ON.	R1-R4	LED changes to OFF once
T4	Hold the button for a long interval.	R2, R4	no repeated toggles occur while held
T5	Release the button after a press.	R5	release is reported without a LED toggle
T6	Apply rapid bounces shorter than the debounce window.	R2, R7	false transitions are ignored
T7	Observe serial monitor and LCD output.	R6	events, periodic status, and LCD state are visible
T8	Inspect source tree organization.	R8	drivers, services, and app logic are separated
T9	Review final scope.	R10	only Part 1 Button-LED behavior is implemented

Table 4: Validation scenarios for Lab 11 Part 1

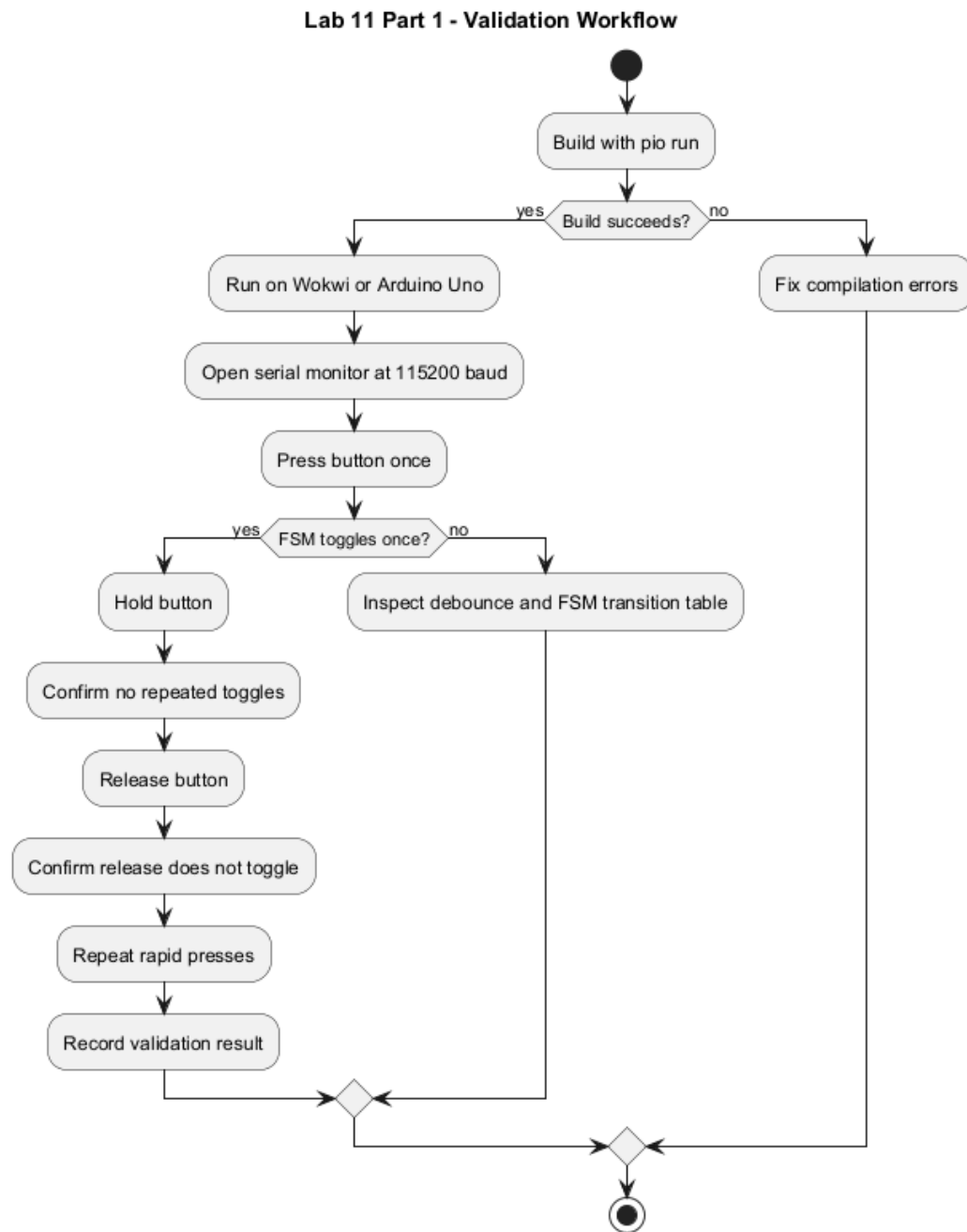


Figure 6: Validation workflow for build and interactive testing

Design phase conclusions

The design phase produced a direct mapping from requirements to implementation modules. The FSM table, debounce flow, and validation scenarios are sufficient to verify the Part 1 behavior without introducing Part 2 traffic-light functionality.

3. Hardware and software implementation

3.1 Electrical configuration

The implementation uses the following Wokwi-compatible wiring:

- button pin D12 connected to one side of the push button;
- the other side of the push button connected to GND;
- D12 configured as `INPUT_PULLUP`, so the logical pressed state is LOW;
- LED anode connected to D9 through a 220 ohm resistor;
- LED cathode connected to GND;
- LCD1602 connected as in Lab 10: RS=D7, E=D6, D4=D5, D5=D4, D6=D3, D7=D13;
- serial USB interface configured at 115200 baud.

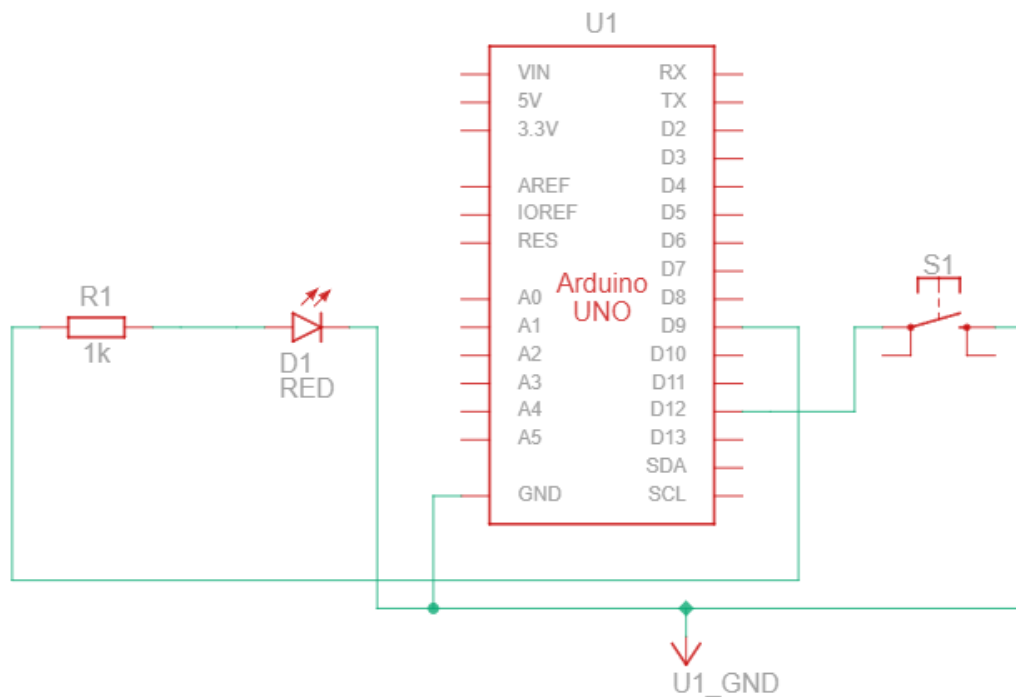


Figure 7: Electrical schema for the Lab 11 Button-LED FSM circuit

Listing 1: Wokwi hardware configuration excerpt

```

1 "connections": [
2   ["uno:9", "r1:1", "green", ["v0"]],
3   ["r1:2", "led1:A", "green", ["v0"]],
4   ["led1:C", "uno:GND.1", "black", ["v0"]],
5   ["btn1:1.r", "uno:12", "green", ["v0"]],

```

```

6  ["btn1:2.r", "uno:GND.2", "black", ["v0"]],
7  ["uno:7", "lcd:RS", "purple", ["v0"]],
8  ["uno:6", "lcd:E", "purple", ["v0"]],
9  ["uno:5", "lcd:D4", "green", ["v0"]],
10 ["uno:4", "lcd:D5", "green", ["v0"]],
11 ["uno:3", "lcd:D6", "green", ["v0"]],
12 ["uno:13", "lcd:D7", "green", ["v0"]]
13 ]

```

The complete simulation files are stored in `Lab11/Code/diagram.json` and `Lab11/Code/wokwi.toml`.

3.2 3.2 Source code organization

The project is organized modularly in `Lab11/Code`:

- `src/main.cpp`: PlatformIO entry point.
- `src/app/app_controller.cpp`: initialization, periodic sampling, event dispatch, and reporting.
- `src/drivers/button_driver.cpp`: active-low button input on D12.
- `src/drivers/led_driver.cpp`: active-high LED output on D9.
- `src/drivers/lcd_display.cpp`: LCD1602 state output using Lab 10 pins.
- `src/drivers/serial_stdio.cpp`: serial redirect for `printf`.
- `src/services/debounce_filter.cpp`: reusable consecutive-sample debounce filter.
- `src/services/button_led_fsm.cpp`: Moore FSM transition table and output logic.

3.3 3.3 Critical implementation sections

FSM transition table

```

1 constexpr FsmStateDefinition kFsmTable[] = {
2     {"LED_OFF", false, {ButtonLedState::LedOff, ButtonLedState::LedOn}},
3     {"LED_ON", true, {ButtonLedState::LedOn, ButtonLedState::LedOff}},
4 };

```

Debounce acceptance logic

```

1 if (filter.counter < filter.requiredSamples) {
2     filter.counter++;
3 }
4
5 if (filter.counter < filter.requiredSamples) {

```

```

6   return false;
7 }
8
9 filter.stableValue = filter.candidateValue;
10 filter.counter = 0U;
11 return true;

```

Press-event handling

```

1 if (debounceFilterStableValue(g_buttonDebounce)) {
2     const ButtonLedState previousState = buttonLedFsmState(g_fsm);
3     buttonLedFsmDispatch(g_fsm, ButtonLedInput::PressEvent);
4     applyFsmOutput();
5     g_toggleCount++;
6     printf("EVENT_t=%lu_ttype=PRESS_from=%s_to=%s_led=%s\n", ...);
7 }

```

Serial STDIO setup

```

1 void serialStdioSetup(uint32_t baudRate) {
2     Serial.begin(baudRate);
3     FILE *serialStream = fdevopen(serialPutChar, serialGetChar);
4     stdin = serialStream;
5     stdout = serialStream;
6     stderr = serialStream;
7 }

```

LCD status output

```

1 LiquidCrystal g_lcd(7, 6, 5, 4, 3, 13);
2 FILE *g_lcdStream = fdevopen(lcdPutChar, nullptr);
3
4 moveToLine(0);
5 fprintf(g_lcdStream, "%-8s_LED:%-2.2s", fsmStateName, ledOn ? "ON" : "OF");
6 moveToLine(1);
7 fprintf(g_lcdStream, "B:%-3.3s_T:%03lu", buttonPressed ? "PRS" : "REL",
8         toggleCount % 1000UL);

```

3.4 3.4 Implementation reasoning

The FSM receives only debounced press events. This avoids the common error where a button held in the pressed state toggles the LED repeatedly. Release transitions are still reported for observability, but they are not sent as FSM toggle inputs.

The application loop is non-blocking. Button sampling and status reporting are scheduled with `millis()`, which keeps the response predictable and avoids busy-wait loops. Text reporting uses standard I/O formatting: terminal messages are emitted through `printf`, and LCD lines are emitted through an LCD-backed `FILE` stream with `fprintf`.

The selected debounce latency is approximately 30 ms, leaving a large margin against the 100 ms requirement.

Implementation phase conclusions

The implementation phase produced a compact but modular Part 1 solution. The source code separates hardware access from FSM and debounce logic, making the core behavior reusable and easier to validate.

4. Testing and validation

4.1 Validation methodology

Validation combines compilation and interactive checks:

1. build the project with `pio run`;
2. open the serial monitor at 115200 baud and observe the LCD;
3. press the button once and verify one state transition;
4. hold the button and verify that no repeated transitions occur;
5. release the button and verify that only a release event is printed;
6. repeat rapid presses to verify debounce behavior;
7. inspect the source tree and confirm that only Part 1 is implemented.

4.2 4.2 Validation results

ID	Check	Status	Observation
T1	Build verification	Pass	PlatformIO build succeeded for Arduino Uno
T2	OFF to ON transition	Ready	verified by FSM table; interactive Wokwi or hardware run confirms LED output
T3	ON to OFF transition	Ready	verified by FSM table; interactive Wokwi or hardware run confirms LED output
T4	Held button behavior	Ready	debounce emits only one stable press transition while held
T5	Release behavior	Ready	release event is printed without dispatching PressEvent
T6	Bounce rejection	Ready	six consecutive equal samples are required before a state change
T7	Serial and LCD reporting	Pass	event/status lines and LCD state output are implemented
T8	Modular structure review	Pass	app, driver, and service modules are separated
T9	Scope review	Pass	no Part 2 traffic-light logic is present

Table 5: Validation summary for Lab 11 Part 1

4.3 4.3 Proof of operation

The project includes a Wokwi-ready electrical configuration. The following evidence should be captured when the simulation or real board is run:

- `terminal.png`: serial monitor output showing **EVENT** and **STATUS** lines;
- `electrical_schema.png`: electrical circuit schema for the Arduino Uno, button, LED, and LCD;
- `hardware.jpg`: optional real hardware assembly photo.

```
Lab 11 - FSM Button-LED, Part 1 only
Button: D12 to GND, active LOW with INPUT_PULLUP
LED: D9 through resistor, active HIGH
LCD1602: RS=D7 E=D6 D4=D5 D5=D4 D6=D3 D7=D13
Debounce: 6 samples x 5 ms = 30 ms
Press the button once to toggle the FSM state.

STATUS t=26 state=LED_OFF led=OFF button=RELEASED toggles=0
STATUS t=526 state=LED_OFF led=OFF button=RELEASED toggles=0
STATUS t=1026 state=LED_OFF led=OFF button=RELEASED toggles=0
EVENT t=1067 type=PRESS from=LED_OFF to=LED_ON led=ON
EVENT t=1116 type=RELEASE state=LED_ON led=ON
EVENT t=1301 type=PRESS from=LED_ON to=LED_OFF led=OFF
```

Figure 8: Serial terminal output during Button-LED FSM operation

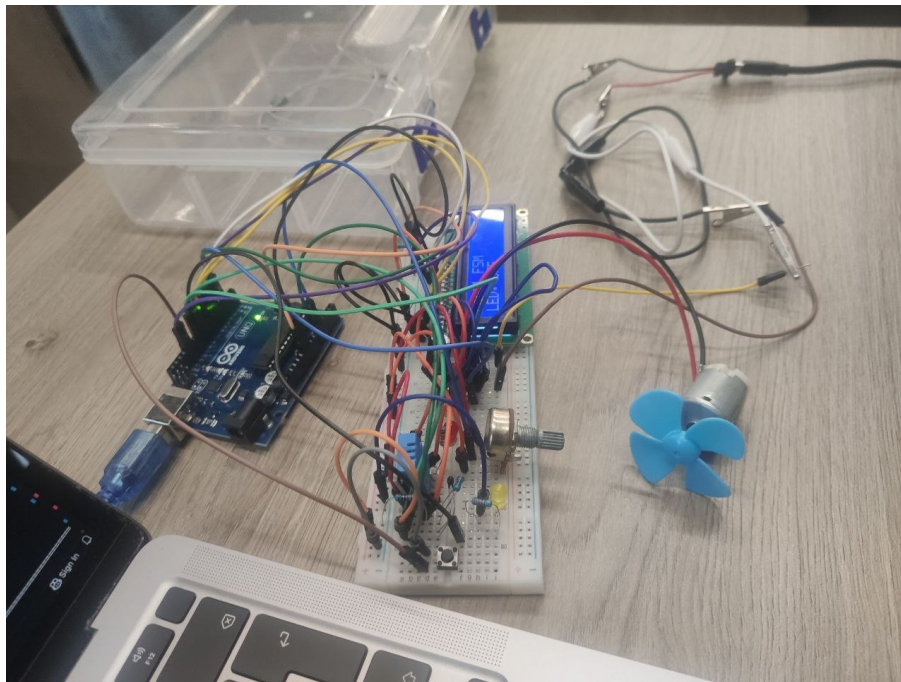


Figure 9: Optional real hardware assembly for Lab 11 Part 1

4.4 Observations and recommendations

The most important implementation detail is the separation between raw button level and FSM input event. The FSM should not receive the raw button state directly, because a long press would then act as repeated input. The debounce filter converts the physical signal into stable transitions, and the application controller forwards only the stable press transition to the FSM.

If the system is extended later, the button and LED drivers can be reused unchanged. Additional FSM states or outputs should be added in the service layer rather than directly in the Arduino loop.

Testing phase conclusions

Compilation validates the buildability of the firmware, and the defined interactive tests cover all functional requirements. The implementation is ready for Wokwi or Arduino Uno execution and remains inside the required Part 1 scope.

General conclusions

The final Lab 11 project implements the requested Lab 7 Part 1 Button-LED FSM. It uses a Moore finite state machine with two states, a software debounce filter, modular drivers, serial STDIO reporting, and LCD1602 state output. Each valid button press toggles the LED once, while release events and contact bounce do not cause unwanted state changes.

The project follows the course documentation structure and includes requirements, component roles, architecture diagrams, behavioral models, implementation excerpts, validation criteria, and Wokwi simulation files. Possible improvements include adding unit tests for the FSM and debounce service, adding a configurable LCD/serial reporting mode, and capturing final serial and hardware screenshots after interactive execution.

AI tool usage note

During the preparation of this report, the author used OpenAI GPT-based tooling for content generation and restructuring. The resulting information was reviewed, validated, and adjusted according to the project requirements before final inclusion.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for behavioral control with finite state machines, Part 1 Button-LED.
3. PlatformIO Documentation, <https://docs.platformio.org/>.
4. Arduino Language Reference, <https://www.arduino.cc/reference/en/>.
5. Wokwi Documentation, <https://docs.wokwi.com/>.
6. PlantUML Language Reference, <https://plantuml.com/>.

7. University of Texas at Austin, Embedded Systems - Shape the World: Designing a FSM.

Appendix

A. Project artifacts

The implementation is stored in `Lab11/Code/`.

B. GitHub repository

Repository URL: `https://github.com/Tirppy/si-course-repo/tree/main/Lab11`

C. Diagram sources

PlantUML source files are stored in `Lab11/Report/diagrams/`. Rendered images are stored in `Lab11/Report/images/`.

D. Build and run commands

- Build: `pio run` from `Lab11/Code`
- Serial monitor: `pio device monitor -b 115200`
- Wokwi simulation: use `Lab11/Code/diagram.json` and `Lab11/Code/wokwi.toml`